ExamForge AI — Observability Report

Complete Enterprise Observability Platform

Generated: 2026-07-25 12:27 UTC

Part A — Crash Reporting

The CrashReporter captures all exception types across the Flutter application: Flutter exceptions (widget build failures, layout errors via FlutterError.onError), Dart exceptions (manual reporting via reportDartException), async exceptions (reportAsyncException), isolate exceptions, platform exceptions (PlatformException via reportPlatformException), fatal errors (reportFatalError), and non-fatal errors (reportNonFatalError). Every crash report is enriched with a correlation ID linking to the distributed tracing system, user ID, school ID, session ID, app version, build number, device information, platform, stack trace, and feature module (auto-extracted from stack trace paths). Reports are buffered (max 200 entries) and logged through the StructuredLogger for immediate visibility. Severity classification uses CrashSeverity (nonFatal, degraded, fatal, critical) based on exception type. PII redaction removes Bearer tokens, password values, and partially redacts email addresses.

Part B — Structured Log Shipping

The LogShippingService extends the existing StructuredLogger to add production log shipping. It supports 10 log levels: INFO, WARNING, ERROR, CRITICAL, SECURITY, AUDIT, AI, EXAM, SYNC, and NETWORK. Each ShippableLogEntry includes 14 mandatory fields: timestamp, correlation_id, user_id, school_id, module, feature, request_id, duration, status, device, platform, version, level, and message. Logs are buffered (max 1000 entries) and shipped in batches (configurable size, default 50) on a timer interval (default 30 seconds). Failed uploads retry with exponential backoff (1s base, max 60s, with jitter to prevent thundering herd). Entries that exhaust all retries move to an offline queue for later shipping when connectivity is restored. Permanently failed entries go to a dead-letter queue. Compression is supported (configurable) via JSON minification. The shipOfflineQueue() method processes accumulated offline entries when connectivity is restored.

Part C — Health Monitoring

The HealthMonitoringService monitors 10 system components: Supabase Auth, Supabase Database, Supabase Realtime, Supabase Storage, Supabase Edge Functions, AI Provider, Notification Service, Offline Engine, Sync Queue, and Background Jobs. Each component is registered and periodically checked (default 60-second interval). Health checks evaluate latency thresholds to determine status: under 500ms = Healthy, 500-1000ms = Degraded, over 1000ms = Critical, unreachable = Offline. The SystemHealthReport aggregates all components, with overall status being the worst individual component status. Consecutive error counts are tracked for escalation decisions. Error messages in health checks are redacted to prevent leaking connection details (Supabase URL, keys).

Part D — Metrics System

The MetricsService tracks operational metrics across 4 types: counters (monotonically increasing values like request/error counts), gauges (point-in-time values like memory/CPU), histograms (distribution of values with percentile statistics), and ratios (percentage values like cache hit ratio). Specific tracked metrics include: API latency per endpoint, AI request latency per operation, database query latency per query type, sync operation

duration, exam submission time, realtime latency, memory usage, CPU usage, battery level, network usage, and cache hit ratio. The LatencyTracker calculates average, p50, p90, p95, and p99 percentiles from up to 1000 samples. All metrics include correlation IDs for linking to distributed traces and tags for dimensional analysis.

Part E — Alert Engine

The AlertEngine evaluates 10 alert categories with configurable thresholds: crash spikes (crashes per minute exceeding threshold), authentication failures (consecutive auth errors), slow APIs (latency exceeding threshold), database timeouts, AI failures (consecutive AI errors), storage failures, realtime disconnects, queue growth (sync queue items exceeding threshold), sync failures (consecutive sync errors), and rate limit spikes. Alert severity has 4 levels: info, warning, critical, emergency. Critical and emergency alerts trigger escalation through 3 tiers: Tier 1 (Team Lead, 5-minute delay, in-app + email), Tier 2 (Engineering Manager, 15-minute delay, in-app + email + SMS), Tier 3 (VP Engineering, 30-minute delay, email + SMS + phone). Thresholds are configurable per environment via AlertThresholds class.

Part F — Distributed Tracing

The TracingService provides distributed tracing with correlation IDs, parent/child span relationships, duration tracking, and status management. Every trace starts with a root span and can have unlimited child spans. Each span tracks operation name, feature module, start/end times, duration, status (started/completed/failed/cancelled), and metadata. Convenience methods provide trace creation for specific domains: traceApiRequest() for HTTP requests, traceAiRequest() for AI operations, traceExamSubmission() for exam submissions, and traceSyncOperation() for sync operations. Active traces are stored by correlation ID, and completed traces are archived (max 200). The DistributedTrace class calculates overall status (worst span status) and total duration. All trace data includes correlation IDs that link to the StructuredLogger context.

Part G — Production Diagnostics

The DiagnosticsService collects a comprehensive snapshot of application state including: app version, build number, git commit (if available), environment (development/staging/production), current user ID, school ID, user role, realtime status, database status, storage status, current AI provider, offline queue length, pending sync count, memory usage, network type, offline status, overall health status, metrics snapshot, crash count, active traces, and active alerts. The DiagnosticInfo class serializes all fields to JSON for API shipping. Diagnostics are disabled in production via MonitoringFeatureFlags for security.

Part H — Background Worker Monitoring

The BackgroundWorkerMonitor tracks worker lifecycle: idle, running, errored, dead status. Job tracking covers: queued, running, completed, failed, cancelled, and dead_letter states. Each JobInfo records ID, type, status, timestamps, retry count, error message, duration, and metadata. Worker registration, status updates, job completion, failure recording, cancellation tracking, and long-running job detection are all supported. Jobs that exceed 3 retries are moved to a dead-letter queue. Average execution time is tracked using incremental averaging. The monitor provides summary statistics: queue size, total failures, dead-letter count, and average execution time.

Part I — Monitoring Dashboard

The MonitoringDashboardData aggregates all observability data into a single object for admin dashboard rendering: system health status, crash count, active alerts (with critical alert count), active and completed trace

counts, API latency statistics, worker status summary, log shipping status, metrics snapshot, and timestamp. Riverpod providers unify data from all 10 observability modules: monitoringDashboardProvider (main aggregation), systemHealthLabelProvider (quick status), totalTraceCountProvider, and alertSummaryProvider (alerts by category). These providers are designed for use in a Flutter admin dashboard UI that would display charts, status indicators, and real-time metrics.

Part J — Production Configuration

The MonitoringConfig provides environment-specific configurations for development, staging, and production. Each config includes: MonitoringFeatureFlags (9 boolean flags controlling which observability features are active), SamplingRates (10 rate values from 0.0 to 1.0 controlling which events are captured), minimum log level (ExtendedLogLevel), LogShippingConfig (batch size, interval, buffer limits, retry settings, compression), AlertThresholds (10 threshold values for alert generation), health check interval, crash buffer limit, and metric buffer limit. Key differences by environment: development captures all events (sampling 1.0) with minimal shipping; staging ships logs with moderate sampling (0.5-0.8); production uses aggressive sampling (0.05-0.5) with compression enabled, diagnostics disabled, and minimum log level set to WARNING. The MonitoringConfig.resolve() method auto-selects the correct configuration based on Flutter build mode (kDebugMode, kProfileMode, kReleaseMode).